How the FLF Epistemic Stack works

Brandon Flores · Doctrine Labs · brandon@bmflores.com

Repo (public): https://github.com/brandon25/doctrine-labs-flf-epistack

What this is: A short, human-written explanation of how the current build works. [illegible] claim. Not a slide deck.

Co-writing (straight)

This note was co-written with Cursor, the coding agent I work with daily. I call it [illegible]
Same division of labor [illegible] Jun 21 [illegible] early feedback [illegible] engineer every piece [illegible]
[illegible] the tests, and revise when something is wrong.
[illegible] Depth on every line is uneven. I'm about six months into softwa[illegible]
sixteen years of industrial design before that. If a line here is wrong, the error is [illegible]

1. The problem (same bug as Jun 21 / Jul 19)

Three claims can cite one brochure and still get counted as three independent s[illegible]

That's the bug.

The early packets shipped a provenance genealogy layer: count roots under a [illegible]
citations [illegible] from the same ultimate source vs novel evidence contribution: distinguishing repeat[illegible]

This build still does that, then goes one step further. It asks whether a conclusi[illegible]
from [illegible] something that cannot share the author's b[illegible]
Navigation plus assessment.

Out of scope (unchanged): settling COVID origins, LHC safety, or egg nutrition.
Infrastructure and evidential support only.

2. What the system does, in four stages

Four stages. Separate modules. Run in order.

Stage 1 Resolve identity (genealogy)

Code: src/epistemic/ingest.js, genealogy.js, source_identity.js

Load a case folder (evidence_blocks.json, source_registry.json). Map every block to three levels:

Level	Counts	Example
Claim	One excerpt	One quote in the JSON
Document	One publication unit	One paper, one report
Lineage	One underlying observation	One experiment, one dataset, one generative event

Headline number is always lineage. Counting claims overstates by how many excerpts you pulled. Counting documents still treats two follow-on papers from one dataset as two sources. Lineage is the conservative count. Overstating independence is the failure mode we care about.

Live shape (COVID, after this build): excerpts documents lineages, with a citation inflation factor printed on the report.

Stage 2 Propose

Code: src/epistemic/adjudicate.js (propose), llm.js

Ask a question of the corpus. A local model (Ollama) returns JSON: conclusion, confidence, optional assumption, and reasoning steps. Each step must cite a block and quote the passage it relies on.

That output is a proposal. Not a verdict.

Stage 3 Verify (deterministic)

Code: verifyCitations() in adjudicate.js, measure.js

No model in this stage. For each reasoning step:

1. Cited block IDs must exist in the corpus.
2. Text quotes must appear verbatim in the cited block.

3. Measurement blocks: re-run a whitelisted read-only command and compare declared value to measured value. Exact match
Cite without quote   flagged weak. All weak   verdict unsubstantiated (not a pass).
If a measurement can't run on this machine (repo not present), result is unverifiable_here.
Third state. Not the same as failed. Unchecked never counts as verified.

Stage 4   Challenge panel + verdict
Code: mechanical_challenge.js, rest of adjudicate.js, model_identity.js
Two seats on the panel.
Seat 1   mechanical challenger. Always runs. Zero models. Eight checks (C1 C8): coverage, lineage span, single-lineage concentration, uncited contradiction, confidence mismatch, weak steps, quote concentration, measurement validity. Shares no weights or training data with the proposer because it isn't a model. Independence is a property of the code, not a calibration. This is the only seat whose independence we can guarantee.
Seat 2   model challenger (optional). Sees the conclusion, not the reasoning (blind). Picked via models/registry.json. Only counts as independent if weight lineage is verified different from the proposer's. Same family = second call, not second source.
Verdicts (computed, not declared):
Verdict • Meaning
unverified • Citation check failed. Done.
unsubstantiated • Every step cited without quoting.
verified_unchallenged • Citations ok, no challenge ran.
verified • Citations ok, challenges found nothing.
verified_with_caveat • Citations ok, a single lineage objected at medium/caveat level (e.g. overstated, not high-severity unsup
contested • Two or more independent lineages objected.
Human is the last lineage on the panel. Recording a position raises independence to strong.
Positions write to decisions/*.json. "On the record" is real, not copy.

3. How that answers the bug
Overcounting. Stage 1 reports lineages, not citation counts. Cochrane's Handbook (Ch. 5)

still says there is no fully automated recommended tool for reconciling multiple reports of one study. We didn't invent that problem. We shipped tooling for the half that stays manual in most workflows.

Unchallenged conclusions. Stages 2 4 force shown work, mechanical citation checks, then a panel that includes a challenger that cannot share the proposer's blind spots. Same-model self-preference is documented (Panickssery et al., NeurIPS 2024). Seat 1 is the structural answer.

Self-check. Case self points the tool at claims about me. Mechanical challenger catches overstatement in my own evidence. Same shape as one lineage counted several times in COVID. docs/DEFECT_REGISTER.md lists bugs I shipped. Every entry names a test. npm run verify:register fails if a claimed guard is missing.

4. What it does / doesn’t (honest)

Does:

- Resolve claim document lineage and print the conservative count
- Require cited, quoted reasoning on every judgment
- Return measurement claims from a whitelist
- Run a model-free mechanical challenge every time
- Persist human positions

Doesn’t:

- Find papers (Elicit)
- Index citation statements at web scale (Scite)
- Verify fabricated/misquoted references at corpus scale (CiteAudit, CiteTracer, and related tools)
- Settle the underlying scientific questions

Uplift claim stays narrow: structure and checkable independence, not better prose and not domain expertise.

5. What’s in the repo now (verify, don’t trust me)

As of this build: npm test 188 passing in the epistack repo (no network). Re-run before review.

Piece	Where
Cover note	`COVER_NOTE_TO_FLF.md`
Methodology	`SPEC.md`
Defect register	`docs/DEFECT_REGISTER.md`
Prior art (incl. blanks in our row)	`docs/PRIOR_ART.md`
Frozen demos (no install)	`docs/transcripts/` (26 runs)
Genealogy + adjudication	`src/epistemic/`
Mechanical checks C1–C8	`src/epistemic/mechanical_challenge.js`

```bash
git clone https://github.com/brandon255/doctrine-labs-flf-epistack.git
cd doctrine-labs-flf-epistack
npm test
npm run epistemic:covid
```

No install beyond Node 20. No API key. Mechanical half runs cold. Local model optional for live propose/challenge. Prefer not to clone: open docs/transcripts/ on GitHub.

6. What I claim / what I don't

What I claim: I found a real gap while dogfooding, built the navigation layer in the early packets, then built the assessment layer on top. It runs. Tests pass. Failures are named. I'm learning the domain in public.

What I do not claim: first to notice the problem (Greenberg 2009; Cochrane handbook). Senior engineer credentials. That every adjacent tool is worse. That COVID/LHC/eggs are settled.

I'm not submitting as an epistemologist. I'm submitting as a builder who found an infrastructure gap while doing FLF's homework, then kept building.

Brandon

Doctrine Labs • co-written with Cursor (Rivet) • Brandon Flores